

✓ ML Fairness and Explainability

Introduction

This assignment focuses on building, analyzing, and interpreting a machine learning model with an emphasis on fairness and explainability. It aims to assess your ability to create models that are ethical, unbiased, and transparent.

✓ Instructions

Dataset Preparation:

- Load any other public dataset relevant to fairness evaluation.
- Preprocess the dataset by selecting features (X) and the target variable (y).
- Identify a sensitive attribute (e.g., gender, age) for fairness analysis.

✓ Dataset Preparation - Implementation

```
# Install necessary libraries
!pip install -q ucimlrepo fairlearn shap lime scikit-learn pandas numpy matplotlib
```

```
import pandas as pd
import numpy as np
from ucimlrepo import fetch_ucirepo

# Fetch the Adult Income dataset
adult = fetch_ucirepo(id=2)

# Data (features) and Target (y)
X = adult.data.features
y = adult.data.targets

# Combine for easier preprocessing, then split again
df = pd.concat([X, y], axis=1)

# Preprocessing for Adult dataset
# Drop rows with missing values (NaN represented as '?')
df = df.replace('?', np.nan).dropna()

# Convert categorical features to numerical using one-hot encoding
df = pd.get_dummies(df, drop_first=True)
```

```
# Rename target column to be consistent (if it was an object type)
if 'income_>50K' in df.columns:
    df['income'] = df['income_>50K'].astype(int)
    df = df.drop(columns=['income_>50K', 'income_<=50K'], errors='ignore') # Drop c
elif 'income_>50K.' in df.columns:
    df['income'] = df['income_>50K.'].astype(int)
    df = df.drop(columns=['income_>50K.', 'income_<=50K.'], errors='ignore')

# Define features (X), target (y), and sensitive attribute
X = df.drop(columns='income')
y = df['income']

# Identify a sensitive attribute (e.g., 'sex_Male')
sensitive_attribute = df['sex_Male'] # Assuming 'sex' is a sensitive attribute and

print("Dataset Head:")
display(df.head())
print(f"Sensitive attribute chosen: sex_Male")
print(f"Shape of features (X): {X.shape}")
print(f"Shape of target (y): {y.shape}")
```

Dataset Head:

	age	fnlwgt	education- num	capital- gain	capital- loss	hours- per- week	workclass_Local- gov	workclass_
0	39	77516	13	2174	0	40	False	
1	50	83311	13	0	0	13	False	
2	38	215646	9	0	0	40	False	
3	53	234721	7	0	0	40	False	
4	28	338409	13	0	0	40	False	

5 rows × 99 columns

Sensitive attribute chosen: sex_Male

Shape of features (X): (45222, 98)

Shape of target (y): (45222,)

Model Training and Evaluation:

- Split the dataset into training and testing sets.
- Train a logistic regression model using scikit-learn.
- Evaluate model performance using accuracy, confusion matrices, and classification reports.

✓ Model Training and Evaluation - Implementation

```

from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, confusion_matrix, classification_report

# Split the dataset into training and testing sets
X_train, X_test, y_train, y_test, A_train, A_test = train_test_split(
    X, y, sensitive_attribute, test_size=0.3, random_state=42
)

# Initialize and train a Logistic Regression model
model = LogisticRegression(solver='liblinear', random_state=42, max_iter=1000)
model.fit(X_train, y_train)

# Predict on the test set
y_pred = model.predict(X_test)

# Evaluate model performance
print("\n--- Model Evaluation ---")
print(f"Accuracy: {accuracy_score(y_test, y_pred):.4f}")
print("\nConfusion Matrix:")
print(confusion_matrix(y_test, y_pred))
print("\nClassification Report:")
print(classification_report(y_test, y_pred))

```

```
--- Model Evaluation ---
```

```
Accuracy: 0.8321
```

```
Confusion Matrix:
```

```
[[11238   62]
 [ 2216   51]]
```

```
Classification Report:
```

	precision	recall	f1-score	support
0	0.84	0.99	0.91	11300
1	0.45	0.02	0.04	2267
accuracy			0.83	13567
macro avg	0.64	0.51	0.48	13567
weighted avg	0.77	0.83	0.76	13567

Fairness Analysis:

- Compute fairness metrics such as selection rate, false positive rate, and true positive rate using Fairlearn.
- Create a MetricFrame to analyze metrics across groups defined by the sensitive feature.
- Visualize fairness metrics using bar plots for better interpretability.

✓ Fairness Analysis - Implementation

```

from fairlearn.metrics import MetricFrame, selection_rate, false_positive_rate, true_positive_rate
import matplotlib.pyplot as plt
import seaborn as sns

# Define the sensitive feature for the test set
A_test_sensitive = A_test.rename('Sensitive Group') # Rename for better plotting later

# Define fairness metrics
fairness_metrics = {
    'selection_rate': selection_rate,
    'false_positive_rate': false_positive_rate,
    'true_positive_rate': true_positive_rate
}

# Create a MetricFrame to analyze metrics across groups
grouped_on_sensitive_feature = MetricFrame(
    metrics=fairness_metrics,
    y_true=y_test,
    y_pred=y_pred,
    sensitive_features=A_test_sensitive
)

print("\n--- Fairness Metrics Across Sensitive Groups ---")
print(grouped_on_sensitive_feature.by_group)

# Visualize fairness metrics using bar plots
fig, ax = plt.subplots(1, 3, figsize=(18, 5))
fig.suptitle('Fairness Metrics by Sensitive Group', fontsize=16)

# Selection Rate
grouped_on_sensitive_feature.by_group.plot.bar(y='selection_rate', ax=ax[0], legend=False)
ax[0].set_title('Selection Rate')
ax[0].set_ylabel('Selection Rate')
ax[0].tick_params(axis='x', rotation=0)

# False Positive Rate
grouped_on_sensitive_feature.by_group.plot.bar(y='false_positive_rate', ax=ax[1], legend=False)
ax[1].set_title('False Positive Rate')
ax[1].set_ylabel('FPR')
ax[1].tick_params(axis='x', rotation=0)

# True Positive Rate
grouped_on_sensitive_feature.by_group.plot.bar(y='true_positive_rate', ax=ax[2], legend=False)
ax[2].set_title('True Positive Rate')
ax[2].set_ylabel('TPR')
ax[2].tick_params(axis='x', rotation=0)

plt.tight_layout(rect=[0, 0.03, 1, 0.95])
plt.show()

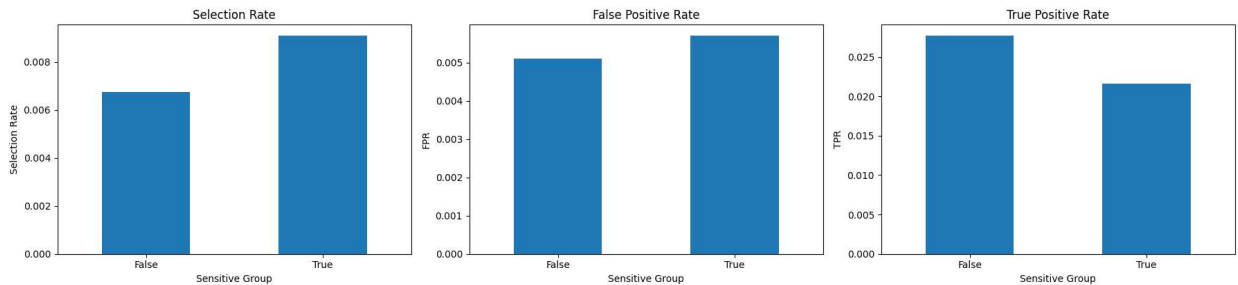
```

```

--- Fairness Metrics Across Sensitive Groups ---
              selection_rate  false_positive_rate  true_positive_rate
Sensitive Group
False              0.006752              0.005100              0.027692
True              0.009097              0.005709              0.021627

```

Fairness Metrics by Sensitive Group



✦ Explainability Analysis:

- Apply SHAP (SHapley Additive exPlanations) for global and local explainability:
 - Generate SHAP summary plots to highlight important features globally.
 - Use SHAP waterfall plots to explain individual predictions locally.
- Use LIME (Local Interpretable Model-agnostic Explanations) for detailed interpretation of individual predictions.

◆ Gemini

```

import lime
import lime.lime_tabular
import numpy as np # Ensure numpy is imported for .values conversion

print("\n--- LIME for Local Explainability ---")

# Set an instance index for explanation
instance_idx = 0

# Initialize LIME Tabular Explainer
# Using training data for feature statistics
feature_names = X_train.columns.tolist()
class_names = ['<=50K', '>50K'] # Assuming income classes for binary classification

lime_explainer = lime.lime_tabular.LimeTabularExplainer(
    training_data=X_train.values,
    feature_names=feature_names,
    class_names=class_names,
    mode='classification'
)

# Explain an individual prediction (using the defined instance_idx)
print(f"Explaining prediction for instance {instance_idx} using LIME...")
# Get the instance and its prediction probability

```

```

# Get the instance and its prediction probability
instance_to_explain = X_test.iloc[instance_idx].values

# Generate explanation
lime_explanation = lime_explainer.explain_instance(
    data_row=instance_to_explain,
    predict_fn=model.predict_proba,
    num_features=5 # Display top 5 contributing features
)

# Print explanation as text
print("LIME Explanation (text):")
print(lime_explanation.as_list())

# Visualize explanation (optional, requires specific notebook setup to render inline
print("Generating LIME explanation plot (opens in new window if not inline)...")
lime_explanation.show_in_notebook(show_all=False)

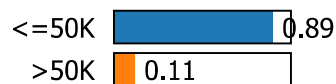
```

```

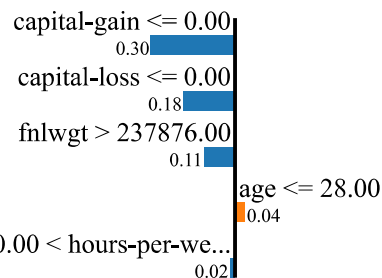
--- LIME for Local Explainability ---
Explaining prediction for instance 0 using LIME...
/usr/local/lib/python3.13/dist-packages/sklearn/utils/validation.py:2739: UserWarning:
  warnings.warn(
LIME Explanation (text):
[('capital-gain <= 0.00', -0.30020972584279737), ('capital-loss <= 0.00', -0.1837302
Generating LIME explanation plot (opens in new window if not inline)...

```

Prediction probabilities



<=50K **>50K**



Feature	Value
capital-gain	0.00
capital-loss	0.00
fnlwgt	319768.00
age	28.00
hours-per-week	45.00

